

【详解】手撕 一维、二维、三维差分数组原理（附图解，模板，例题分析）

原创 已于 2023-03-16 13:05:00 修改 · 8.9k 阅读 · 62 点赞 · 109 收藏 · CC 4.0 BY-SA 版权

文章标签: #算法 #数据结构 #c语言 #c++ #开发语言



2048 AI社区 文章已被社区收录



算法基础知识 专栏收录该内容

11 篇文章

【差分专题】

引言

差分是一种处理数据巧妙而简单的方法，可以应用于区间修改和询问问题。例如，将给定的数据集 合 A 分成很多区间，并对这些区间进行很多次操作对某段区间内的所有元素做相同的加减操作，此时若一个个的遍历修改这段区间内的每个元素，就会非常的耗时，效率不高。为此，我们引入一个叫做 B，当修改这段区间时，只需要对这段区间的端点进行修改，就能将整段区间的元素修改，这个操作是非常平滑的，只需要 $O(1)$ 的复杂度。当我们作进行完毕后，再利用差分数组，即可计算出新的数据集 A。

对于集合 A，它可以是一维的线性数组 $a[]$ ，二维矩阵 $a[][]$ ，三维立体 $a[][][]$ 。与之对应的，差分数组 $B[]$ ， $B[][]$ ， $B[][][]$ 。当然如果想象维也是有可能的。

1. 一维差分

1.1 基本概念

问题描述：给定一个长度为 n 的一维数组 $a[]$ ，数组内的每个元素都有初值，并对其进行一下操作：

- (1) **修改操作：**进行 m 次区间修改，每次修改对这段区间 $[L, R]$ 内的所有元素做相同的加减操作。
- (2) **查询操作：**查询某个元素的值是多少

如果简单的暴力遍历，那么对于每次的修改复杂度是 $O(n)$ 的，有 m 次修改，此时复杂度为 $O(n*m)$ ，如果用差分法的话，复杂度将为 $O(n + m)$

原理

对于这个原数组 $a[] = \{a_1, a_2, a_3, \dots, a_n\}$ ，我们构造出这样一个数组 $B[] = \{b_1, b_2, b_3, \dots, b_n\}$ ，使得 $a_i = b_1 + b_2 + \dots + b_i$ ，那么 $b[]$ 就称为 $a[]$ 称为 $b[]$ 的前缀和。可以发现，差分与前缀和是一组逆运算。

如何利用差分数组对区间进行修改呢？为什么利用差分数组能提升修改的效率呢？

1.区间修改，时间复杂度为 $O(1)$

现在要将原数组 $a[]$ 区间 $[L, R]$ 上的每个数都加上一个 c，如下图所示：

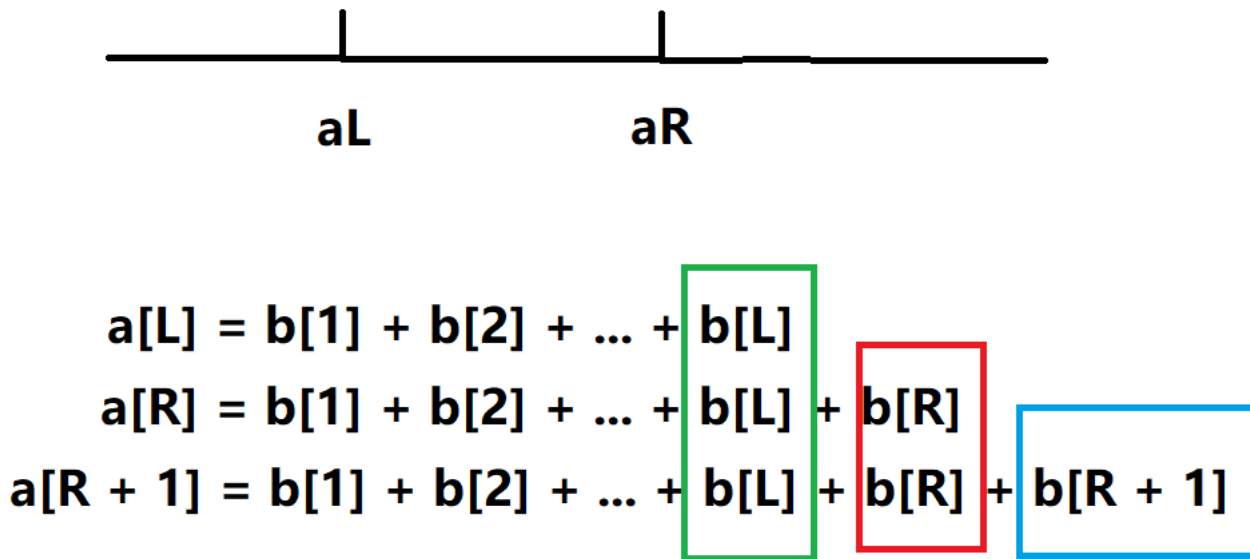


NothingAtall.

关注

62





- 第一个受到影响的差分数组中的元素为 $b[L]$ ，所以 $b[L] += c$ ，对于 $a[L]$ 后面的数都会受到 $b[L]$ 的影响加上 c 。
- 最后一个受影响的差分数组中的元素 $b[R]$ ，为了保证不影响到 R 后面的元素，所以我们需要 $b[R + 1] -= c$ 。

也就是说，对于 $a[x] = b[1] + b[2] + \dots + b[x]$ ，利用差分数组能够精确地实现只修改指定区间内元素的目的，而不会修改区间外的 $a[]$ 的值，也就是：

- (1) $1 \leq x < L$ ，前缀和 $a[x]$ 不变。
- (2) $L \leq x \leq R$ ，前缀和 $a[x]$ 加上了 c 。
- (3) $R < x \leq N$ ，前缀和 $a[x]$ 不变，因为被 $b[R + 1]$ 中的 c 抵消了。

这样一来，就不必每次都对区间内所有的数进行处理，只需要修改区间 $[L, R]$ 的两个端点 $b[]$ 的值即可，复杂度是 $O(1)$ 的。

c 一键获取完整项目代码

```
1 void add(int l, int r, int c)
2 {
3     b[l] += c;
4     b[r + 1] -= c;
5 }
```

2.单点查询，时间复杂度为 $O(n)$

根据定义，差分数组 $b[x]$ 的前缀和 $b[1] + b[2] + \dots + b[x]$ 就是原数组 $a[x]$ 的值

c 一键获取完整项目代码

```
1 void get(int x)
2 {
3     for(int i = 1; i <= n; i++)
4         sum[i] = sum[i - 1] + b[i];
5
6     cout << sum[x];
7 }
```

初始化：我们其实不需要过分关注差分数组 $b[]$ 是如何构造出来的，只需要知道差分与前缀和是互为逆运算即可。实际上，我们压根就不需要去构造差

一开始，我们就可以把原数组 $a[]$ 看成全是 0 ，即 $a[] = \{0, 0, 0, \dots, 0\}$ ，此时相应的差分数组 $b[]$ 也全是 0 ，即 $b[] = \{0, 0, 0, \dots, 0\}$ ，用组 $a[]$ 的初始值：

- $a[1]$ 相当于区间 $[1, 1]$ 的每个数都加上 $a[1]$
- $a[2]$ 相当于区间 $[2, 2]$ 的每个数都加上 $a[2]$
- ...

NothingAtAll. 关注

62

- $a[n]$ 相当于区间 $[n, n]$ 的每个数都加上 $a[n]$

这样，只需要用上面的区间修改操作 `add()` 即可完成初始化 赋值了。

1.2 例题分析

【例题1】区间(interval)

题目描述：小明喜欢数组。他现在有一个 n 个元素的数组 a ，而他要对其 $a[L] \sim a[R]$ 进行 M 次操作：

操作一：将 $a[L] \sim a[R]$ 内的元素都加上 c

操作二：将 $a[L] \sim a[R]$ 内的元素都减去 c

最后询问 $a[L] \sim a[R]$ 内的元素之和

AcCode

c

一键获取完整项目代码

```
1 #include<bits/stdc++.h>
2
3 using namespace std;
4 typedef long long LL;
5 const int N = 1000010;
6
7 LL a[N];
8
9 void add(int l, int r, int c)
```

展开

差分的局限性：我们可以注意到，利用差分数组 $b[]$ 可以将原来 $O(n)$ 的区间修改，降为 $O(1)$ 的端点修改，从而提高了修改操作的效率。

但是，对于一次的查询操作，我们必须计算前缀和 $b[1] + b[2] + \dots + b[x]$ 才能将原数组 $a[x]$ 求出，计算量是 $O(n)$ 的，即一次查询的复杂度是 $O(n)$ 的。说，如果查询操作发生多次，例如 m 次修改， k 次查询，且修改和查询的顺序是随机的，即可能边修改边查询。此时总复杂度为： m 次修改复杂度 $O(1)$ 复杂度 $O(kn)$ ，即 $O(m + kn)$ 。还不如直接暴力来的快 $O(mn + k)$ 。

可以看出，尽管差分数组对于“区间修改”很高效，但是对于“单点查询”来说略显吃力。此时有专门的数据结构来解决这一类问题：树状数组和线段树，的树状数组和线段树专题。

2. 二维差分

1.1 基本概念

有了一维差分的认识后，我们容易就能拓展到二维差分。一维是线性的，一段区间 $[L, R]$ 有两个端点；二维是一个矩阵，一块区间由四个端点所围

问题描述：在 $n \times n$ 的格子中有 m 个地毯。给出这些地毯的信息，问每个点被多少地毯覆盖。

输入：第一行是两个整数 n, m 。接下来 m 行，每行 2 个坐标 $(x1, y1)$ 和 $(x2, y2)$ ，代表一块地毯，左上角是 $(x1, y1)$ ，右下角是 $(x2, y2)$ 。

输出：输出 n 行，每行 n 个正整数。第 i 行第 j 列的正整数表示 (i, j) 这个格子被多少地毯覆盖。

可以发现，这是前面例题区间 (interval) 的二维拓展，其修改和查询操作完全一样。

我们知道存储矩阵往往需要很大的空间。如果题目有空间的限制，例如 $100M = 100 * 1024 * 1024$ 个字节(byte)，那么对于矩阵每个元素是 4 个字节的可以计算出最大的 $maxn = 5120$ 。不过，也可以像前面例题一样，不定义差分矩阵 $b[][]$ ，直接将原矩阵 $a[][]$ 看成自己的差分矩阵，这样一来就能剩了。

同前面一样，我们先考虑能不能直接暴力求解。可以看出，每次矩阵修改的复杂度是 $O(n^2)$ ，共 m 次，总复杂度为 $O(m + n^2)$ ，肯定会 TLE。

(1)二维差分的定义

在一维差分中，原数组 $a[]$ 是从第 1 个 $b[1]$ 开始的差分数组 $b[]$ 的前缀和： $a[x] = b[1] + b[2] + \dots + b[x]$ 。

在二维差分中， $a[i][j]$ 是差分数组 $b[i][j]$ 的前缀和，即将原点坐标 $(1, 1)$ 和坐标 (i, j) 围成的矩阵中，所有的 $b[i][j]$ 相加等于 $a[i][j]$ 。我们可以把每个 $b[i][j]$ 小格；在坐标 $(1, 1)$ 和 (i, j) 所围成的范围内，所有小格子加起来的总面积，等于 $a[i][j]$ 。如下图中，每个格子的面积是一个 $b[i][j]$ ，例如阴影格 $[i][j]$ ，它由 4 个坐标点组成： (i, j) 、 $(i - 1, j)$ 、 $(i, j - 1)$ 、 $(i - 1, j - 1)$ 。坐标点 (i, j) 的值是 $a[i][j]$ ，它等于坐标 $(1, 1)$ 和 (i, j) 所围成的所有格子

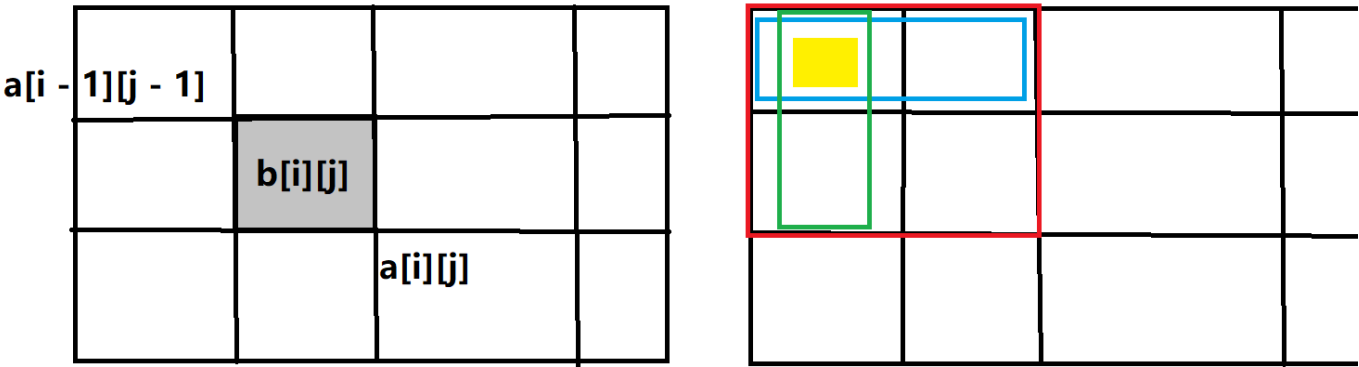


NothingAtAll.

关注

62





把每个a[]看总面积，把每个b[]看成小格子的面积

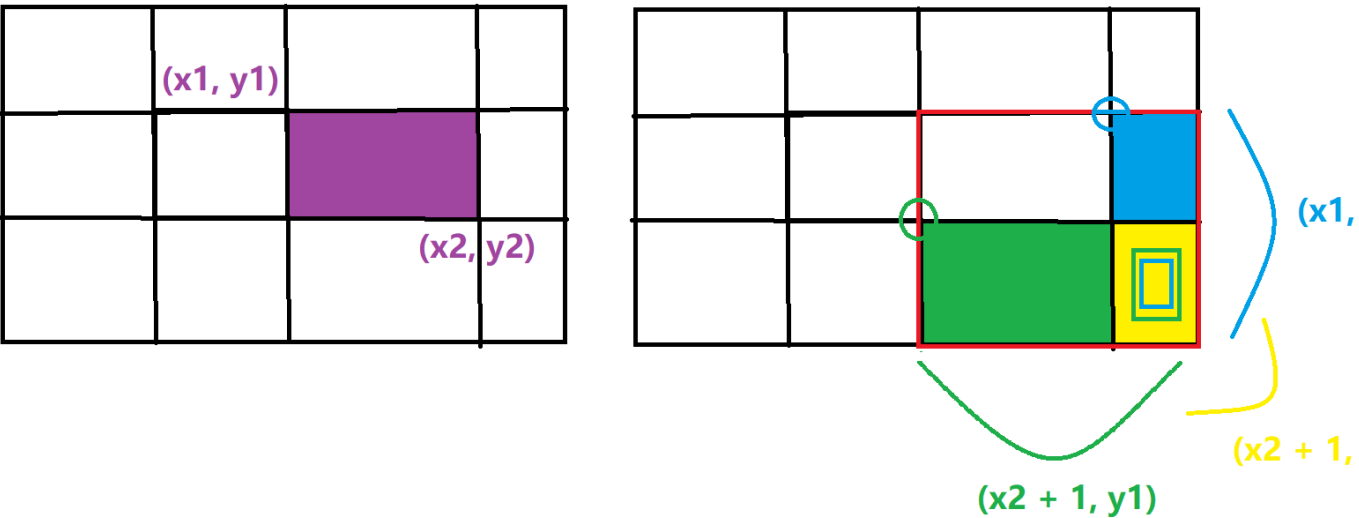
由上图我们可以得到二维差分的定义：在二维情况下，差分就变成了相邻a[][]的"面积差"，计算公式是：

$$b[i][j] = a[i][j] - a[i-1][j] - a[i][j-1] + a[i-1][j-1]$$

即利用上图红色大面积a[i][j]减去两个小面积a[i-1][j]、a[i][j-1]，由于两个小面积公共的部分a[i-1][j-1]被减去了2次，故要加回来1次a[i-1][j-1]

(2)二维区间修改

对于一维区间修改的操作，我们只需要修改区间的两个端点的b[]值。那么相应地，在二维情况下，一块区间是一个矩阵，由4个端点，只需要修改即可。如下图所示，



当我们对坐标点 (x1, y1) ~ (x2, y2) 所围成的区间进行修改时，对应的4个端点的操作应为：

```
c
1 | b[x1][y1] += c; // 二维区间的起点
2 | b[x1][y2 + 1] -= c; // 把 x看成常数, y从 y1 到 y2
3 | b[x2 + 1][y1] -= c; // 把 y看成常熟, x从 x1 到 x2
4 | b[x2 + 1][y2 + 1] += c; // 由于前面两式把 c 减去了 2 次, 故要加回 1 次
```

1.2 例题分析

【例题1】Monitor



NothingAtall.

关注

62



题意：Xiaoteng 有一个 $n \times m$ 的矩形庄稼地，为了抓到小偷，安装了 p 个监控，每个监控都有一个矩形的监视范围，左上角为 $(x1,y1)$ ，右下角为 $(x2,y2)$ 偷 q 次，每次小偷们的作案地点都是一个矩形区域，左上角为 $(x1,y1)$ ，右下角为 $(x2,y2)$ 。问每次小偷们作案时，能否看到全部的小偷。

思路：将每个监控的矩形监视区域里的每个数都加上 1，都操作在差分数组上。求差分数组的前缀和得到原数组，如果原数组中的值大于 1，说明该点被盖，我们只需要记 1 个即可。对于小偷们每次作案的矩形区域，看监控区域是否全部覆盖（是否全是 1），如果全部覆盖（作案矩形同监控矩形的值相等）YES，否则，输出NO。

AcCode

c一键获取完整项目代码

```
1 #include<bits/stdc++.h>
2
3 using namespace std;
4 typedef long long LL;
5
6 int main()
7 {
8     int n, m;
9 }
```

展开

3.三维差分

1.1 基本概念

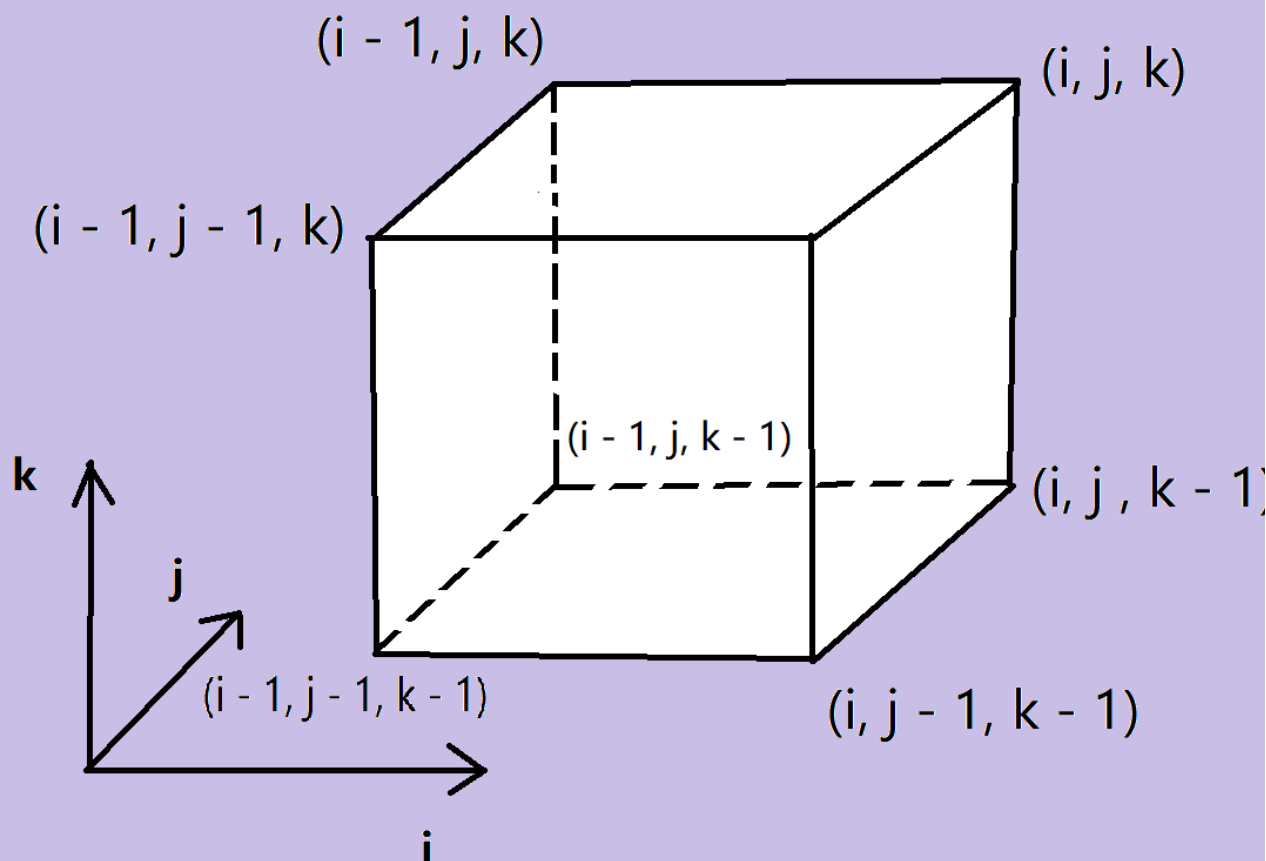
三维已是人类空间想象的一个大跨度，其差分难度较为复杂，不过没关系，下面我们将利用空间立体图来逐一理解。

(1)三维差分的定义

元素值用三维数组 $a[i][j][k]$ 来定义，差分数组 $b[i][j][k]$ 也是三维的。与之前低维度的差分类似，把三维差分想象成立体空间的操作。与之对应的小立点，所以三维的区间需要修改 8 个 $b[i][j][k]$ 的值。

在二维差分中， $a[i][j]$ 是差分数组 $b[i][j]$ 的前缀和，即原点坐标 $(1, 1)$ 和 坐标 (i, j) 围成的矩阵面积。

在三维差分中， $a[i][j][k]$ 是差分数组 $b[i][j][k]$ 的前缀和，即原点坐标 $(1, 1, 1)$ 和 坐标 (i, j, k) 围成的立体体积。同样地，我们把每个 $b[i][j][k]$ 看成一个坐标 $(1, 1, 1) \sim (i, j, k)$ 所围成的空间中，所有小立体加起来的总体积即为 $a[i][j][k]$ 。如下图所示，每个小立方体由 8 个端点定义。坐标点 $a[i][j][k]$ ；图中小立方体的体积是差分数组 $b[i][j][k]$ 的值。



类似的，在三维情况下，差分就变成了相邻的 $a[i][j][k]$ 的“体积差”。那么如何来写出差分的递推计算公式呢？

观察前面一、二维的前缀和我们可以发现，其前缀和规律十分吻合 **容斥原理**。

【一维】： $s[i] = s[i - 1] + a[i];$

【二维】： $s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + a[i][j]$

即对于 **维度为 t** 的前缀和，记 $S(t)$ 为其前缀和的递推式，则我们有：

$$S(t) = a[t] + \sum_{n=1}^{\infty} (-1)^{(n-1)} S([t-1] \text{ 的组合形式}), \quad n \text{ 为 } -1 \text{ 的个数}$$

所以对于三维的差分数组 $b[i][j][k]$ ，其递推式如下：

$$b[i][j][k] = s[i][j][k] - s[i-1][j][k] - s[i][j-1][k] - s[i][j][k-1] + s[i-1][j-1][k] + s[i-1][j][k-1] + s[i][j-1][k-1] - s[i-1][j-1][k-1]$$

我们发现当维度为 t 的时候容斥的**时间复杂度**是 2^t ，而前缀和的总时间复杂度为 $O(n^t 2^t)$ ，即随着维度的升高，时间复杂度增大的很快，不过是可以 $O(n^t t)$ 的，但在此不展开讨论，因为在算法竞赛中很少遇到3维以上的前缀和，而对于 $t \leq 3$ 时 $O(n^t 2^t)$ 与 $O(n^t t)$ 差别并不大，有兴趣的可自行查阅资料。

(2)三维区间修改



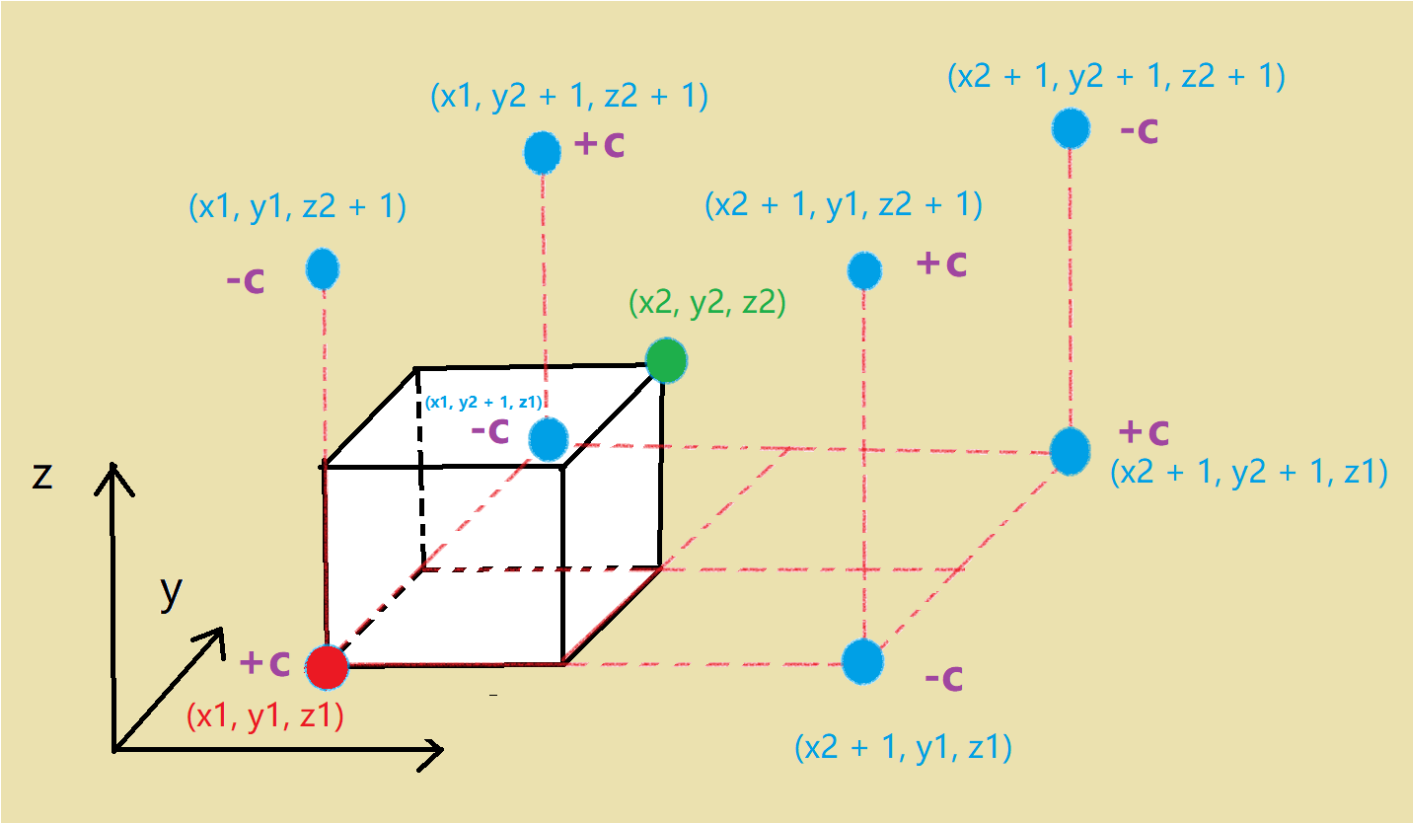
NothingAtAll.

关注

62



在三维情况下，我们修改的是一个立方体，有8个顶点，故我们只需要修改这8个顶点的 差分数组 `b[][][]` 的值即可。给出坐标点 $(x1, y1) \sim (x2, y2, z2)$ 间，如下图所示



三维差分空间图示

那么对应的 8个 `b[][][]` 的修改如下：

c一键获取完整项目代码

```
1 // 前面
2 b[x1][y1][z1] += c; // 坐标起点
3 b[x2 + 1][y1][z1] -= c; // 右下顶点的右边一个点
4 b[x1][y1][z2 + 1] -= c; // 左上顶点的上面一个点
5 b[x2 + 1][y1][z2 + 1] += c; // 右上顶点的斜右上方一个点
6
7 // 后面
8 b[x1][y2 + 1][z1] -= c; // 左下顶点的后面一个点
9 b[x2 + 1][y2 + 1][z1] += c; // 右下顶点的斜右后方一个点
```

展开

可以发现坐标偏移加 1 都与终点相关，为了方便记忆，我们可以把它按照二进制的排列来写，若二进制中出现偶数个 `正1` 为正，奇数个 `正1` 为负，与前缀和是偶数个 `负1` 为负，奇数个 `负1` 为正）：

c一键获取完整项目代码

```
1 {0, 0, 0, 1} // x1, y1, z1
2 {0, 0, 1, -1} // x1, y1, z2 + 1
3 {0, 1, 0, -1} // x1, y2 + 1, z1
4 {0, 1, 1, 1} // x1, y2 + 1, z2 + 1
5
6 {1, 0, 0, -1} // x2 + 1, y1, z1
7 {1, 0, 1, 1} // x2 + 1, y1, z2 + 1
8 {1, 1, 0, 1} // x2 + 1, y2 + 1, z1
9 {1, 1, 1, -1} // x2 + 1, y2 + 1, z2 + 1
```

1.2 例题分析

【例题1】三体攻击（蓝桥杯2018年省赛 A组）

【题目描述】：

 **NothingAtAll.** 关注

 62 

三体人将对地球发起攻击。为了抵御攻击，地球人派出了 $n = A \times B \times C$ 艘战舰，在太空中排成一个 A 层 B 行 C 列的立方体。其中，第 i 层第 j 行第 k 列（记为战舰 (i, j, k) ）的生命值为 $s(i, j, k)$ 。

三体人将会对地球发起 m 轮“立方体攻击”，每次攻击会对一个小立方体中的所有战舰都造成相同的伤害。具体地，第 t 轮攻击用 7 个参数 $x1, x2, y1, y2, z1, z2, d$ 描述；所有满足 $i \in [x1, x2], j \in [y1, y2], k \in [z1, z2]$ 的战舰 (i, j, k) 会受到 d 的伤害。如果一个战舰累计受到的总伤害超过其防御力，那它就会爆炸。

地球指挥官希望你能告诉他，第一艘爆炸的战舰是在哪一轮攻击后爆炸的。

【输入】：

第一行包括 4 个正整数 A, B, C, m；

第二行包含 $A \times B \times C$ 个整数，其中第 $((i - 1) \times B + (j - 1)) \times C + (k - 1) + 1$ 个数为 $s(i, j, k)$ ；

第 3 到第 m + 2 行中，第 (t - 2) 行包含 7 个正整数 $x1, x2, y1, y2, z1, z2, d$ 。 $A \times B \times C \leq 10^6, m \leq 10^6, 0 \leq s(i, j, k), d \leq 10^6$

【输出】：

输出第一个爆炸的战舰是在哪一轮攻击后爆炸的。保证一定存在这样的战舰。

分析：如果对于每次攻击我们都枚举每个点生命值，对于 $n = 10^6$ 个点， $m = 10^6$ 次攻击来说，时间复杂度为 $O(nm)$ ，肯定会TLE。

故此时我们需要在差分数组查询空间内每个元素是否小于0，但是计算空间上的每个元素的值是 $O(n)$ 的，对于m次攻击，复杂度仍然是 $O(nm)$ 。可见数组是不行的。为此我们需要利用二分法：题目保证一定存在这样一个战舰在某一轮攻击后会爆炸，即从第 1 次到 m 次修改，必定有一次是临界值。之前，战舰的生命肯定是大于 0 的，在临界值后，若战舰出现了负值，则之后一直都会是负值。那么对 m 进行二分，就能在 $O(\log m)$ 次内找到这个临界值。总复杂度为 $O(n \log m)$ 。

细节处理：如果对于数组空间我们每一维度都开到 10^6 ，必定会MLE。为了避免这一问题，我们将三维数组拍平为一维数组，即采用压维的方式，对于 10^6 ，数组长度仍然是 10^6 的。对于二维数组【A】【B】，若知道某一元素的下标 (i, j) ，则将它映射为一维数组的下标应为 $i \times B + j$ 。同样对于三维数组【A】【B】【C】，先把前两维看成一维的，再把第三维度看成二维，则三维坐标 (i, j, k) 对应的映射坐标为 $(i \times B + j) \times C + k$ ，实现函数为 get()。

AcCode

c一键获取完整项目代码

```
1 #include<bits/stdc++.h>
2
3 using namespace std;
4 typedef long long LL;
5 const int N = 2000010;
6
7 int A, B, C, m; // 层, 行, 列, 攻击轮数
8 LL s[N], b[N], bp[N]; // 生命值, 差分数组, 备份差分数组
9 int op[N / 2][7]; // 攻击范围及伤害
```

展开

以上内容尚未完全，随着今后学习的推进，我会继续对其进行补充与完善。另外，大家如果觉得我写的还行的话，还请赠予我一个可爱的赞，你的赞对我支持。

CLIP 计算过程图解

CLIP模型是OpenAI开发的一种多模态学习模型，它通过学习文本和图像之间的关联，实现了跨模态的语义理解。

MOS晶体管单元电路的最佳二维公共质心布局生成

在本文中，提出了一种在二维公共质心 MOS 晶体管矩阵中拟合任意沟道宽度晶体管的通用算法。所提出的算法保证了晶体管单元电路的布局不仅是完整的公共质心，而且

55 条评论



纤维Fibre 热评 我想请问一下，这里a*b*c<=1e6,为什么define N为2000010呢？我总是觉得大小为(a+1)(b+1)(c+1)，不知道哪里想错了

二维前缀和&二维差分(超详细,python版,其他语言也很轻松能看懂)-CSDN...

递推公式有了就可以构造差分矩阵,为了方便理解和书写,定义矩阵时要多定义一行和一列(差分数组要多定义两行两列,之后区间加减数时会用到,使之不超出索引范围) 差分矩阵

差分数组详解与应用

思路:我们开始时假设所有牛的高度都想等为0。当我们被告知A和B 可以相互看见时,考虑到题目要求每头牛的身高最可能大,那么就把他们中间的所有牛的身高都减1,那这时候

【详解】手撕 一维、二维、三维差分数组原理（附图解

2401_8416467

/ 由于前面两式把 c 减去了 2 次，故要加回 1 次。// 把 x 看成常数，y 从 y1 到 y2。// 把 y 看成常熟，x 从 x1 到 x2。// 二维区间的起点。

差分 (plus++)

一篇讲得很好的二维差分 二维差分 如果扩展到二维，我们需要让二维



NothingAtAll.

关注

supreme56

62



【蓝桥杯 算法】---第三讲--差分_二维差分数组
一维数组的差分 二维数组的差分 概念 在引出差分的概念之前,先看如下的两个数组 其差分数组为: 可以观察到a数组其实是b数组的前缀和 $a[1]=b[1]$ $a[2]=b[1]+b[2]$ $a[3]=b[1]+b[2]+b[3]$

...一维、二维、三维差分数组原理(附图解(1),2024年最新大数据开发0基础...
求差分数组的前缀和得到原数组,如果原数组中的值大于 1,说明该点被多个监控覆盖,我们只需要记 1 个即可。对于小偷们每次作案的矩形区域,看监控区域是否全部覆盖(是否:

算法数学---差分数组（Difference Array） 最新发布 MzKyle
差分数组是一种高效处理区间更新的数据结构，通过记录相邻元素的差值将区间更新复杂度从 $O(n)$ 降至 $O(1)$ 。其核心原理是通过差分数组两端点进行加减操作（ $D[L]+v$, $D[R]$
差分数组 weixin_3073630
今天学习了一下差分数组，觉得好神奇啊。 1.定义： 对于已知有n个元素的离线数列d，我们可以建立记录它每项与前一项差值的差分数组f：显然， $f[1]=d[1]-0=d[1]$;对于整数

【差分】详解二维前缀和和差分问题_二维数组前缀和
1. 二维前缀和 首先来看下二维前缀和,假设现在有下列的二维数组: 现在设定 $sum[i][j]$ 表示以 (i,j) 为右下角顶点,以 $(0,0)$ 为左上角顶点的矩形的总和,比如 $sum[0][0] = 1, sum[1][1]$
二维差分和三维差分 一名蒟蒻
二维差分（差分与前缀和的下标都从1开始，避免出现越界） 算法公式：在 $x1, y1, x2, y2$ ，区间内+x就是 $sum[x1][y1]+=w$ 、 $sum[x2+1][y1]-=w$ 、 $sum[x1][y2+1]-=w$ 、 $sum[x2+1][y2+1]+=w$
二维差分 weixin_3043748
二维差分和一维差分思路上并没有什么区别，具体实现的区别就在于一维的直接对区间两端差分就好了，而二维的多了一维需要处理。 差分的思想是和前缀和有关的

差分数组模板 greatcode
差分 差分构造方式 原数组A， 差分数组B， 数组范围【1， n】 $B[1] = A[1] - A[0] = A[1]$ $B[i] = A[i] - A[i-1]$ 差分特点 B数组的前缀和 <> A数组的元素值 $SUM(B[1~i]) = A[i]$ A数组
图文详解二维差分 热门推荐 melonyzzz
图文详解二维差分。

二维差分---三维差分算法笔记 小青菜
二维差分和三维差分

二维全息谱教育指南：如何将屈梁生院士的理论应用于教学
在信息技术飞速发展的今天，二维全息谱理论作为一种新兴的信息处理和展示技术，在教育、工业、医疗等多个领域展现出巨大的应用潜力。该技术的核心是将传统的信息展

差分数组（模板） qq_4486696
#include<bits/stdc++.h> //s为原数组， b为差分数组 int s[1010],b[1010]; using namespace std; int main(){ int n,m; cin>>n>>m; for(int i=1;i<=n;i++) cin>>s[i]; //存放原数组 //构造差分数组

差分 数组 qq_4188352
今天在做leetcode每日一题的时候，又学到了一个特殊的数组《差分数组》。链接。

差分数组的总结 LjcoderDCC
差分数组的资料网上不太好找，蒟蒻表示网上学习这个自认为比较简单的技巧是比较麻烦的，于是蒟蒻觉得自己写下学习总结。 博客上看拉个题目意思大概是： 给定一个长

数据结构：差分数组 weixin_3078229
差分数组不仅仅是一个优秀的数据结构，还是一种很好的思想 差分数组的功能是修改区间， 查询点 修改区间的时间复杂度是 $O(1)$ ， 查询点的时间复杂度为 $O(n)$ 我们这!

关于我们 招贤纳士 商务合作 寻求报道 400-660-0108 kefu@csdn.net 在线客服 工作时间 8:30-22:00

公安备案号11010502030143 京ICP备19004658号 京网文〔2020〕1039-165号 经营性网站备案信息 北京互联网违法和不良信息举报中心 家长监护 网络110报警服务 中国互联网举报中心 Chrome商店下载 账号管理规范 版权与免责声明 版权申诉 出版物许可证 营业执照

©1999-2025北京创新乐知网络技术有限公司



NothingAtall.

博客等级 码龄4年

28

258

624

525

原创

点赞

收藏

粉丝

关注

私信

热门文章

数据流图（DFD）这么理解吗？ 16754

乘法逆元（Inverse element）及四大相关求法详解（含证明） 11101

【详解】手撕 一维、二维、三维差分数组原理（附图解，模板，例题分析） 8934



NothingAtall.

关注

62

树状数组详解（附图解，模板及经典例题分析） 6820

手撕迪杰斯特拉算法（Dijkstra）及代码（附图解） 4237

分类专栏



软件工程思考

8篇



计算机网络

1篇



算法基础知识

11篇



C# & Unity

展开全部

上一篇：线段树详解（附图解，模板及经...

下一篇：软件工程是什么？它是不是教会...

最新评论

数据流图（DFD）这么理解吗？

I671671: 话说考试的时候是要全画还是只画顶层和0层

大家在看

ajax的基本使用(上传文件)

Swift 6.2 列传（第十三篇）：香香公主的“倾城之恋”与优先级飞升

C#索引器

CAPL学习-CAN相关函数-CANdb API类函数

硬件+软件协同交付怎么落地？2025 软硬件项目管理工具对比

最新文章

数据流图（DFD）这么理解吗？

谈谈你用过的编程语言和编辑工具

谈谈我了解的测试工具

2023年 8篇 2022年 20篇

目录

【差分专题】

引言

1. 一维差分

1.1 基本概念

1.2 例题分析

2. 二维差分

1.1 基本概念

1.2 例题分析

3.三维差分

1.1 基本概念

1.2 例题分析

收起 ^

 **AtomGit AI** 新一代人工智能开源社区

企业级模型推理API Token 0/0万

企业级模型推理API服务限免使用 [领取更多](#)

 **DeepSeek-V3.1-Base** 热门

DeepSeek-V3.1 是一款支持思考模式...

文本生成  243  10824

限时领取Token



NothingAtall.

关注

 62

